UNIT-II
IoTandM2M

Syllabus: IOT and M2M: M2M to IoT – A Basic Perspective– Introduction, Differences and similarities between M2M and IoT, SDN and NFV for IoT, M2M Value Chains, IoT Value Chains, An emerging industrial structure for IoT, international driven global value chain and global information monopolies. IOT Architecture: IoT Architecture components, Comparing IoT Architectures, A simplified IoT Architecture, core IoT functional stack, IoT data management and compute stack

2.1. Introduction to M2M-A Basic perspective:

Machine to Machine (M2M) refers to networking of machines (or devices) for the purpose of remote monitoring and control and data exchange.

· Term which is often synonymous with IoT is Machine-to-Machine (M2M).

· IoT and M2M are often used interchangeably.

Fig.2.1 Shows the end-to-end architecture of M2M systems comprises of M2M area networks, communication networks and application domain.

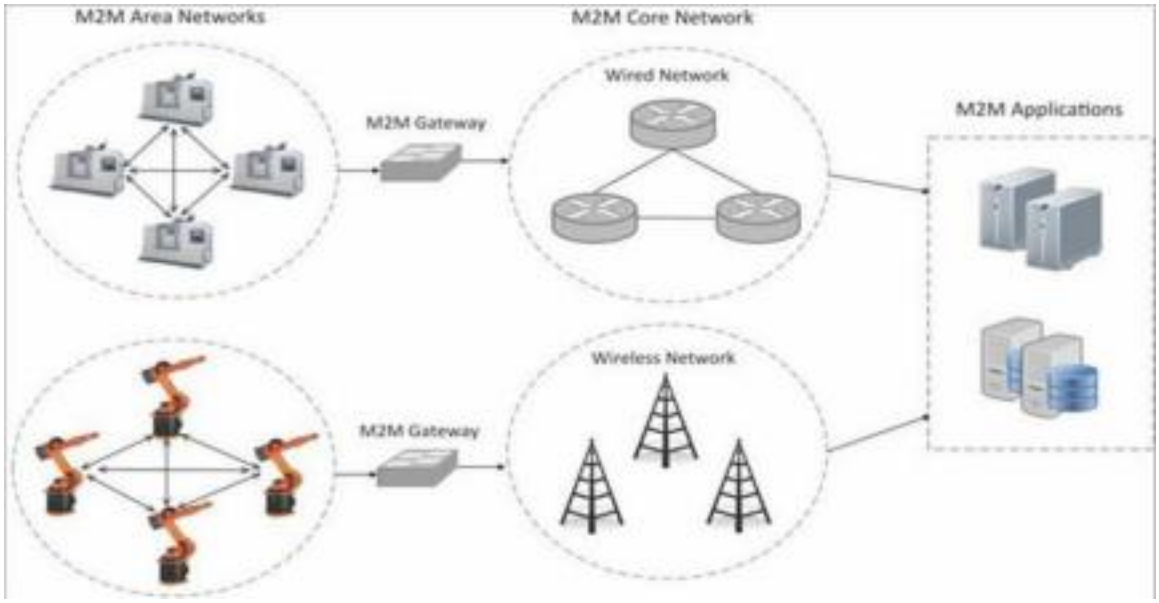


Fig.2.1:M2M Architecture

· An M2M area network comprises of machines (or M2M nodes) which have embedded network modules for sensing, actuation and communicating various communication protocols can be used for M2M LAN such as ZigBee, Bluetooth,

M-bus, Wireless M-Bus etc., These protocols provide connectivity between M2M nodes within an M2M area network.

· The communication network provides connectivity to remote M2M area networks. The communication network provides connectivity to remote M2M area network. The communication network can use either wired or wireless network (IP based). While the M2M are networks use proprietary or non-IP based communication protocols, the communication network uses IP-based network. Since non-IP based protocols are used within M2M area network, the M2M nodes within one network cannot communicate with nodes in an external network.

· To enable the communication between remote M2M are network, M2M gateways are used.

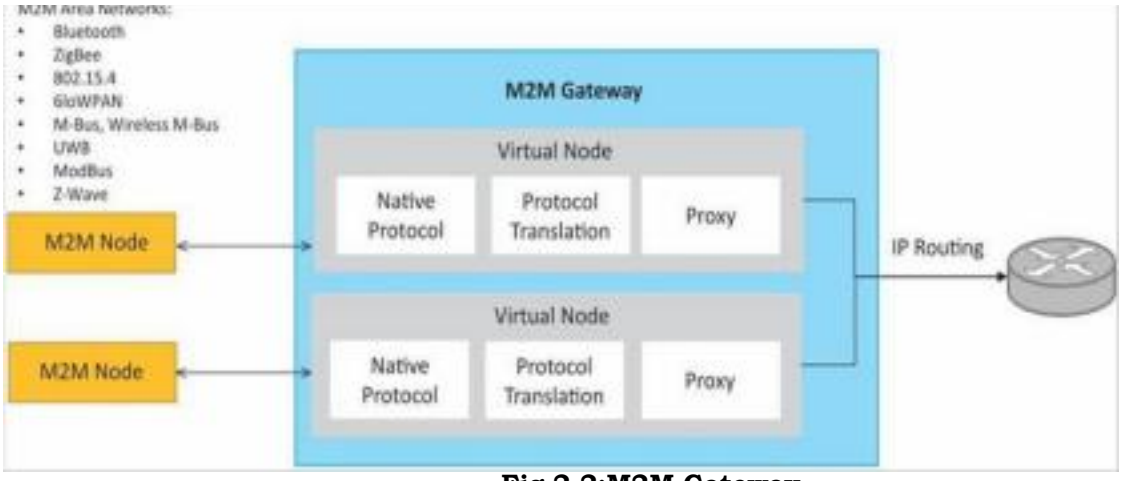


Fig.2.2:M2M Gateway

Fig.2.2. Shows a block diagram of an M2M gateway. The communication between M2M nodes and the M2M gateway is based on the communication protocols which are native to the M2M area network. M2M gateway performs protocol translations to enable Ip-connectivity for M2M area networks. M2M gateway acts as a proxy performing translations from/to native protocols to/from Internet Protocol (IP). With an M2M gateway, each node in an M2M area network appears as a virtualized node for external M2M area networks.

2.2. Differences and Similarities between M2M and IoT

2.2.1. Differences between M2M and IoT

Aspect	M2M	IoT
Definition	Direct communication between two machines/devices, typically point-to-point	Network of interconnected devices communicating over the internet, often through cloud platforms
Communication type	Point-to-point (device to device), often via proprietary or hardwired links	Device to cloud, device to device, and device to gateway — networked and internet-based
Internet dependency	Not necessarily internet-based — can use direct links, cellular, or private networks	Relies heavily on internet connectivity (IP-based networking)
Scale	Usually limited/isolated systems (e.g., a vending machine reporting stock to a central server)	Massive scale — millions of interconnected devices globally
Data sharing	Data is usually used by a single application or a closed system	Data can be shared and accessed by multiple applications/stakeholders via the cloud
Protocols used	SNMP, proprietary protocols, cellular (GSM/GPRS)	HTTP, MQTT, CoAP, WebSocket, and other internet-standard protocols
Analytics/intelligence	Limited — mainly data transfer, minimal processing at the machine level	Strong emphasis on cloud-based analytics, AI/ML, big data processing
Human interaction	Minimal to none — machines act independently	Often includes human interfaces (apps, dashboards) for monitoring/control
Examples	ATM reporting cash levels to bank server, vending machine inventory reporting	Smart home systems, wearable fitness trackers, smart city infrastructure, connected cars
Interoperability	Limited — devices often communicate within a specific closed system	High — designed to work across various platforms and device types

2.2.2. Similarities Between IOT and M2M

- Both involve machines/devices communicating with each other without constant human intervention.
- Both use sensors and communication technology to collect and transmit data.
- Both aim to automate processes and enable data-driven decision-making.

- Both can use similar underlying communication protocols (cellular, Wi-Fi, Zigbee, etc.)
- IoT is often considered an evolution/superset of M2M — they share the same foundational concept of machines exchanging data.

2.2.3. Machines in M2M vs Things in IoT:

• The "Things" in IoT refers to physical objects that have unique identifiers and can sense and communicate with their external environment (and user applications) or their internal physical states.

• M2M systems, in contrast to IoT, typically have homogeneous machine types within an M2M area network.

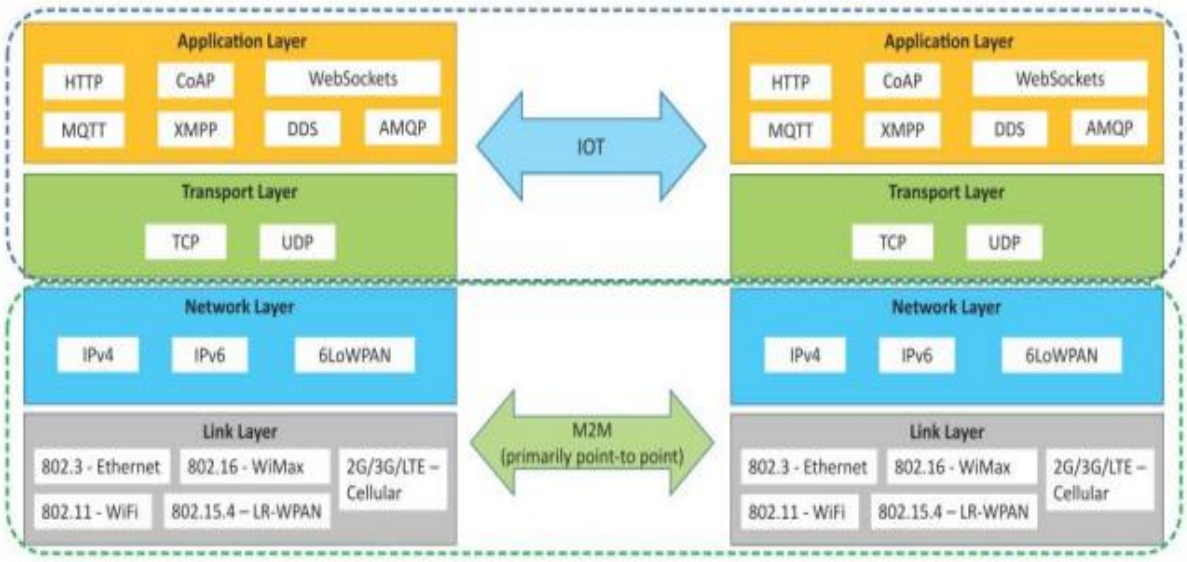


Fig2.3: IoT vs. M2M Architecture Mapping

Hardware vs Software Emphasis:

• While the emphasis of M2M is more on hardware with embedded modules, the emphasis of IoT is more on software.

2.3. Software Defined Networking (SDN):

Software-Defined Networking (SDN) is a network architecture approach that separates the control plane (decision-making) from the data plane (packet forwarding) and centralizes network control in a software-based controller.

Why SDN? (Problems with Traditional Networks)

- In traditional networks, each router/switch makes independent forwarding decisions using embedded control logic.
- This makes network management complex, inflexible, and hard to scale.
- SDN solves this by decoupling control and data planes and introducing centralized, programmable control.

2.3.1.SDN Architecture:

SDN consists of three logical layers:

1. Application Layer (Application Plane)
   - Contains network-aware applications (e.g., load balancers, firewalls, traffic monitoring).
   - Defines *what* the network should do (policies, services).
   - Communicates with the control layer via Northbound APIs.
2. Control Layer (Control Plane)
   - Houses the SDN Controller (e.g., OpenDaylight, ONOS, Ryu).
   - Translates application requirements into flow rules for switches.
   - Maintains a global view of the network topology and state.
   - Communicates with infrastructure layer via Southbound APIs.
3. Infrastructure Layer (Data Plane)
   - Consists of physical/virtual switches and routers that forward packets.
   - Uses simple packet-forwarding hardware (no complex control logic).
   - Receives instructions from the controller via protocols like OpenFlow.

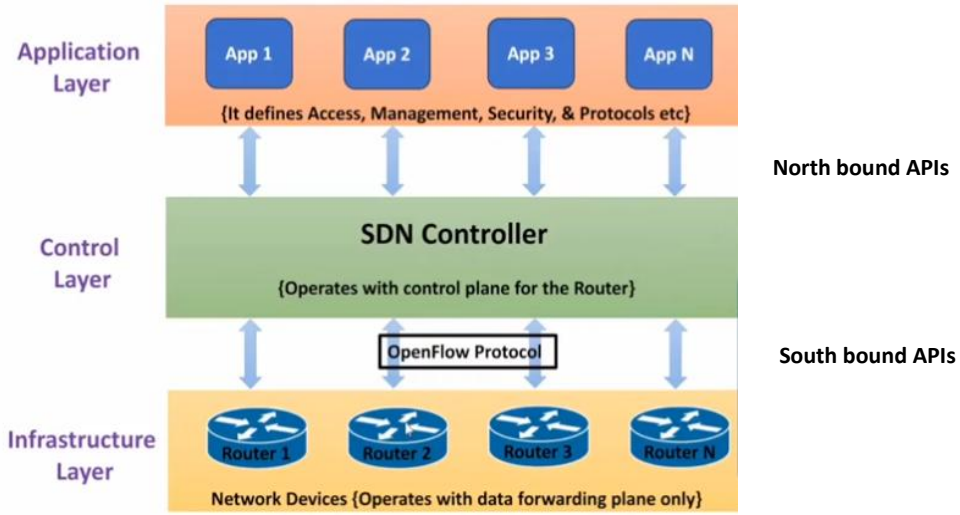


Fig 2.4: Architecture of SDN

Features of SDN

- Centralized Control: Single controller has a global network view.
- Programmability: Open APIs allow custom network applications.
- Dynamic Configuration: Policies can be changed in real-time.
- Vendor Independence: Uses open standards (e.g., OpenFlow).

Advantages of SDN

- Simplifies network management and configuration.
- Enables rapid deployment of new services.
- Reduces dependency on expensive proprietary hardware.
- Improves network agility and flexibility.

Disadvantages of SDN

- Single point of failure (controller).
- Scalability challenges in very large networks.
- Security concerns (centralized control is a target).

2.3.2.SDN for IoT:

Why SDN in IoT?

IoT networks face unique challenges:

- Heterogeneity: Diverse devices (sensors, actuators, gateways) with different protocols.
- Scalability: Millions of devices generating massive traffic.
- Dynamic Topology: Devices frequently join/leave the network.
- Resource Constraints: Limited battery, computation, and memory on IoT devices.
- Security: IoT devices are vulnerable to attacks.

SDN addresses these by providing centralized control, programmability, and dynamic policy enforcement.

2.3.3.SDN-IoT Architecture

The SDN-IoT architecture typically has three layers:

1. Infrastructure Layer
   - Contains IoT devices (sensors, actuators, RFID readers), gateways, and access points.

- Devices connect via base stations, wireless access points, and network gateways.
- Forwards data based on rules from the SDN controller.

2. Control Layer
   - SDN controller manages IoT traffic and enforces policies.
   - Divides the network into discrete subnets for better management.
   - Provides APIs for developers to build IoT applications.
3. Application Layer
   - IoT applications (smart home, healthcare, industrial monitoring).
   - Uses Northbound APIs to interact with the controller without worrying about underlying network details.

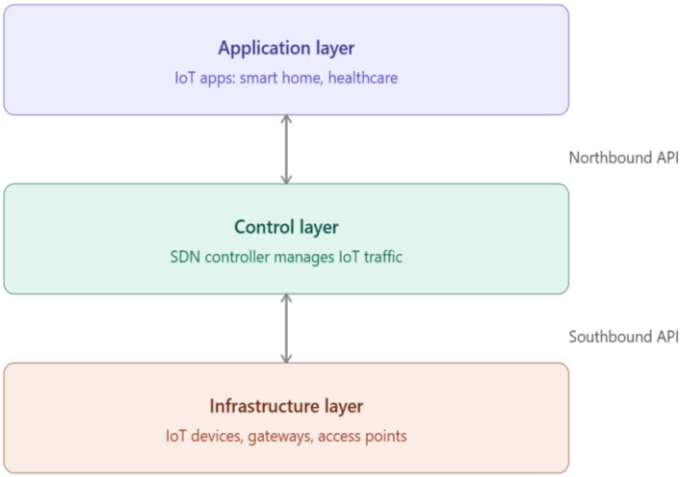


Fig.2.5: SDN-IoT Architecture

2.3.4. Benefits of SDN for IoT

- Centralized Management: Simplifies control of heterogeneous IoT devices.
- Dynamic Policy Enforcement: Security policies can be updated in real-time.
- Scalability: SDN controllers can manage large-scale IoT deployments.
- Edge Integration: SDN can push control to edge servers for low-latency applications.
- Security: SDN can isolate compromised devices and enforce security policies dynamically.

Use Cases of SDN in IoT

- Smart Cities: Traffic management, environmental monitoring.
- Healthcare: Remote patient monitoring with QoS guarantees.
- Industrial IoT: Real-time control of machinery with SDN-based network slicing.
- Smart Home: Centralized control of home automation devices.

2.4. Network Functions Virtualization (NFV)

Network Functions Virtualization (NFV) is a network architecture concept that virtualizes network functions (e.g., firewalls, load balancers, routers) that traditionally ran on dedicated hardware appliances, and runs them as software on commodity servers.

Why NFV? (Problems with Traditional Networks)

- Traditional networks use dedicated hardware for each function (router, firewall, load balancer).
- This leads to high capital expenditure (CapEx) and operational expenditure (OpEx).
- Scaling requires purchasing new hardware, which is slow and expensive.
- NFV replaces hardware appliances with software-based Virtual Network Functions (VNFs) running on standard servers.

2.4.1.NFV Architecture (ETSI Framework)

NFV architecture consists of three main components:

1. NFV Infrastructure (NFVI)
   - Physical Layer: Hardware resources (compute, storage, network).
   - Virtualization Layer: Hypervisor/container engine that abstracts physical resources.
   - Virtual Resources: Virtual machines or containers running VNFs.
2. Virtual Network Functions (VNFs)
   - Software implementations of network functions.
   - Examples: vFirewall, vRouter, vLoadBalancer, vIDS.
   - Run on virtualized infrastructure instead of dedicated hardware.
3. NFV Management and Orchestration (MANO)

- NFV Orchestrator (NFVO): Manages lifecycle of network services (NS) and onboards Network Service Descriptors (NSD).
- VNF Manager (VNFM): Manages lifecycle of individual VNFs (scaling, healing, updates).
- Virtualized Infrastructure Manager (VIM): Allocates and releases NFVI resources (compute, storage, network) upon request.

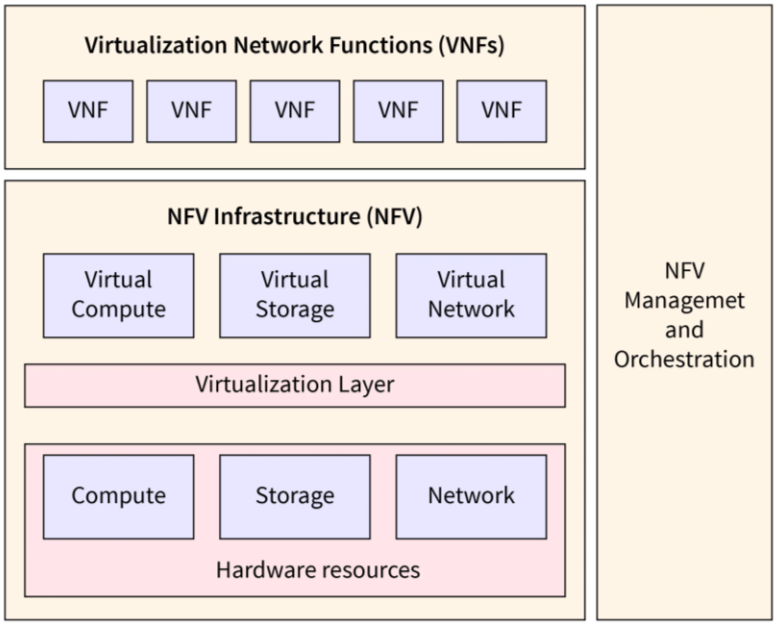


Fig.2.6: Architecture of NFV

Features of NFV

- Hardware Independence: VNFs run on commodity servers, not proprietary appliances.
- Elastic Scaling: VNFs can be scaled up/down on demand.
- Service Chaining: Multiple VNFs can be chained to create complex network services.
- Multi-tenancy: Multiple virtual networks can share the same physical infrastructure.

Advantages of NFV

- Cost Reduction: Eliminates need for expensive dedicated hardware.
- Scalability & Agility: VNFs can be deployed and scaled quickly.
- Faster Time-to-Market: New services can be deployed as software updates.

- Resource Efficiency: Better utilization of hardware through virtualization.

Disadvantages of NFV

- Performance Overhead: Virtualization introduces latency compared to hardware.
- Complexity: MANO framework adds management complexity.
- Security Concerns: Shared infrastructure increases attack surface.

2.4.2. NFV for IoT:

IoT networks require flexible, scalable, and cost-effective network services.

- Scalability: IoT deployments can grow rapidly; NFV allows on-demand scaling of network functions.
- Cost Efficiency: Reduces hardware costs by virtualizing network functions.
- Edge Deployment: NFV enables deploying network functions (e.g., firewalls, analytics) close to IoT devices at the edge.
- Service Customization: Different IoT applications (smart home, healthcare, industrial) require different network services; NFV allows custom service chains.

2.4.3.NFV-IoT Architecture

NFV-IoT architecture integrates NFV components with IoT infrastructure:

1. Physical Layer
   - IoT sensing devices (sensors, actuators).
   - Network forwarding devices (switches, routers).
   - NFV infrastructure (servers, storage).
2. Virtualization Layer
   - Hypervisor/container engine that abstracts physical resources.
   - Enables running VNFs on commodity hardware.
3. VNF Layer
   - Virtualized network functions tailored for IoT:
   - vFirewall (security for IoT traffic)
   - vRouter (routing between IoT subnets)
   - vLoadBalancer (distributing IoT traffic)

- vIDS/IPS (intrusion detection for IoT)
- vAnalytics (data processing at edge)

4. MANO Layer
   - Manages lifecycle of VNFs and network services.
   - Coordinates with SDN controller for traffic steering.
5. Application Layer
   - IoT applications (smart city, healthcare, industrial monitoring).
   - Requests network services via MANO.

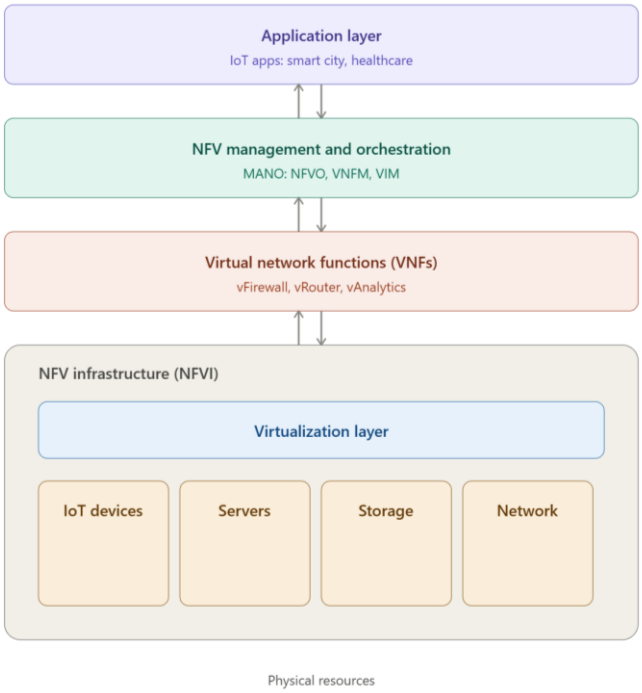


Fig.2.7: NFV-IoT Architecture

2.4.4. Benefits of NFV for IoT

- Cost Reduction: Eliminates need for dedicated hardware at edge locations.
- Scalability: VNFs can be scaled up/down based on IoT traffic demand.
- Edge Computing: Network functions (e.g., analytics, security) can be deployed close to IoT devices for low latency.
- Service Flexibility: Different IoT applications can have custom network service chains.
- Resource Efficiency: Multiple VNFs can share the same physical infrastructure.

Use Cases

- Smart Cities: Virtual firewalls and load balancers for traffic management.
- Healthcare: vIDS/IPS for securing patient data from IoT medical devices.
- Industrial IoT: vAnalytics for real-time data processing at the edge.
- Smart Home: vRouter and vFirewall for home network security.

2.5. SDN Vs NFV

Feature	**SDN**	**NFV**
Focus	Separates control plane from data plane; centralizes network control.	Virtualizes network functions; runs them as software on commodity servers.
Primary Goal	Centralized, programmable network control.	Replace dedicated hardware appliances with software-based functions.
Protocol	Uses OpenFlow for southbound communication.	No specific protocol; uses standards from ETSI NFV.
Implemented at	Data centers, enterprise networks.	Service providers, telecom operators.
Key Benefit	Simplifies network management; enables dynamic policy enforcement.	Reduces hardware costs; increases scalability and agility.
Example Applications	Load balancing, traffic monitoring, QoS enforcement.	vFirewall, vRouter, vLoadBalancer, vIDS.
Business Initiator	Corporate IT departments.	Telecom service providers/operators.

2.6.M2M Value Chains

Machine-to-Machine (M2M) refers to direct communication between devices using wired or wireless channels, without human intervention. M2M value chains describe the stages involved in delivering an M2M solution—from hardware/software development to service delivery.

Stages of M2M Value Chain

1. Inputs: Raw data from M2M devices (e.g., sensor readings).
2. Production/Manufacture: Data is verified, tagged, and prepared.

3. Processing: Aggregation of multiple data sources to create an information component.
4. Packaging: Data is combined with internal corporate databases and made recognizable (e.g., dashboards, reports).
5. Distribution/Marketing: Information products are used internally for decision-making (e.g., maintenance scheduling, product design).

Characteristics

- Vertical and siloed: Focused on one company, one solution.
- Proprietary: Devices and protocols are often custom-built.
- Limited reuse: Data is not shared across solutions or companies.

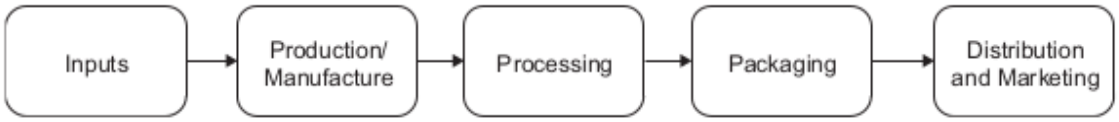


Fig.2.8: M2M Value Chains

2.7. IoT Value Chains

Internet of Things (IoT) extends M2M by connecting devices to the broader Internet, enabling data reuse across solutions and companies. IoT value chains involve more diverse inputs and stakeholders.

Stages of IoT Value Chain

1. Inputs:
   - Devices/Sensors (similar to M2M)
   - Open Data (e.g., government maps, weather data)
   - OSS/BSS (telecom operator data)
   - Corporate Databases (ERP, CRM, etc.)
2. Production/Manufacture: Raw inputs are developed into information components (e.g., asset info, network info).
3. Processing: Data from various sources is mixed and analyzed.
4. Packaging: Information components are created (charts, knowledge frameworks).
5. Distribution/Marketing: Information products are used internally or resold to other economic actors.

Characteristics

- Horizontal and collaborative: Data is reused across value chains.
- Open standards: Uses open APIs and Internet technologies.
- Ecosystem-driven: Multiple companies contribute to value creation.

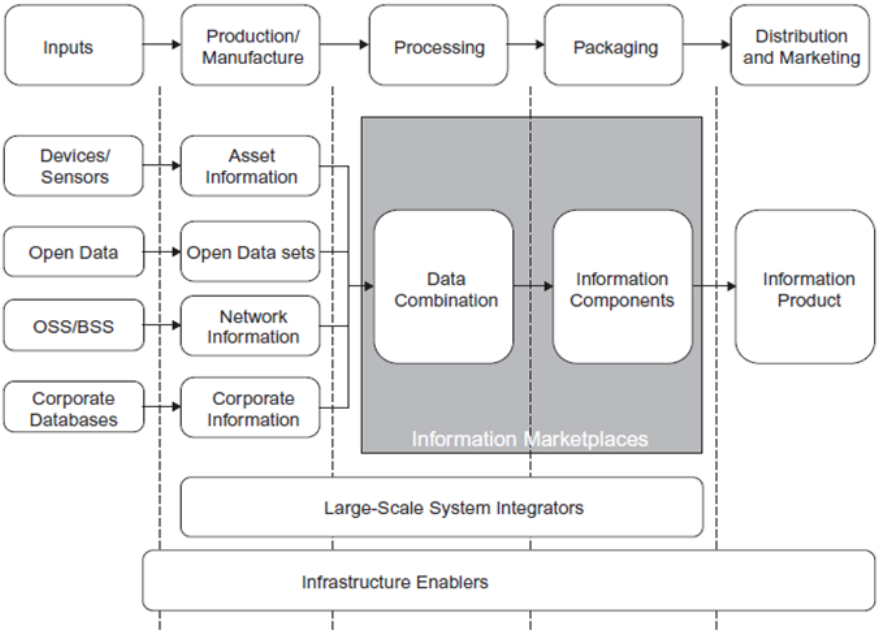


Fig.2.9: IoT Value Chains

2.7.1. M2M vs IoT Value Chains

Feature	M2M Value Chain	IoT Value Chain
Scope	Internal, single company, single solution	Cross-company, multi-solution
Data Reuse	Limited, not shared	Extensive, shared across ecosystems
Standards	Proprietary	Open APIs, Internet protocols
Architecture	Vertical silos	Horizontal, layered
Business Model	hardware/connectivity focused	service/recurring revenue focused
Innovation	Low, vendor lock-in	High, developer ecosystems

2.8. Emerging Industrial Structure for IoT

2.8.1. From silos to ecosystems

- M2M industrial structure: vertical, proprietary, single-solution value chains inside firms.
- IoT industrial structure: horizontal platforms, open APIs, ecosystems with many complementors (device makers, platform providers, app developers, data analytics firms, integrators).

2.8.2. Typical IoT ecosystem roles

- Device & Sensor Manufacturers: broad range of smart devices.
- Connectivity Providers: diverse networks (cellular, LPWAN, Wi-Fi), often with cloud/edge.
- Platform Providers: IoT platforms for device management, data ingestion, analytics.
- Application Developers: vertical solutions (smart city, health, industry).
- System Integrators: integrate multi-vendor devices/services into end-to-end solutions (technical + financial scale matters).
- Data/Analytics Firms: turn raw data into insights; may operate marketplaces.

2.8.3. Why new integrators?

- Technical complexity: many devices, protocols, and services must be integrated.
- Financial scale: only large players/partnerships can recoup R&D and capture integration value; others risk becoming component suppliers.

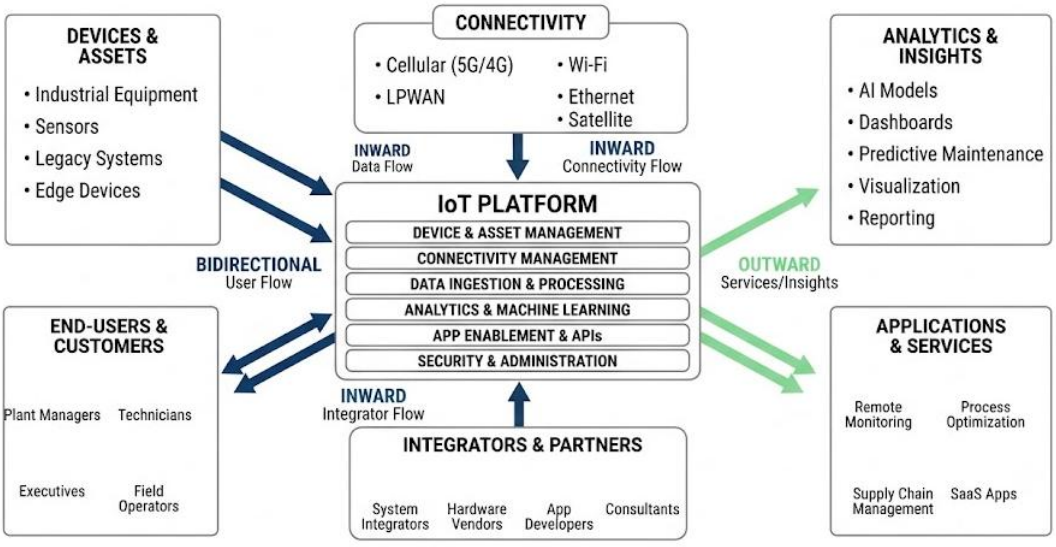


Fig.2.10: Emerging Industrial Structure for IoT

2.9. Information-Driven Global Value Chain (I-GVC)

2.9.1. Core idea

- A new type of global value chain where the "product" is information itself, created from sensor/RFID data plus human-generated digital traces (smartphones, searches, social media).
- Actors coalesce around five fundamental roles in the I-GVC.

2.9.2. Five roles in the I-GVC

1. Inputs:
   - Sensors/RFID and other devices (physical world data).
   - End-users (searches, location, clicks, profiles).
2. Data Factories:
   - Entities producing structured digital data (e.g., map agencies, data vendors).
3. Service Providers / Data Wholesalers:
   - Aggregate and wholesale data streams; often operate at scale (operators, cloud providers).
4. Intermediaries:
   - Anonymize, govern privacy/consent, reduce transaction costs, specialize by niche datasets.
5. Resellers:
   - Combine multiple data sources, analyze, and sell targeted information products (e.g., ad-tech firms like BlueKai).

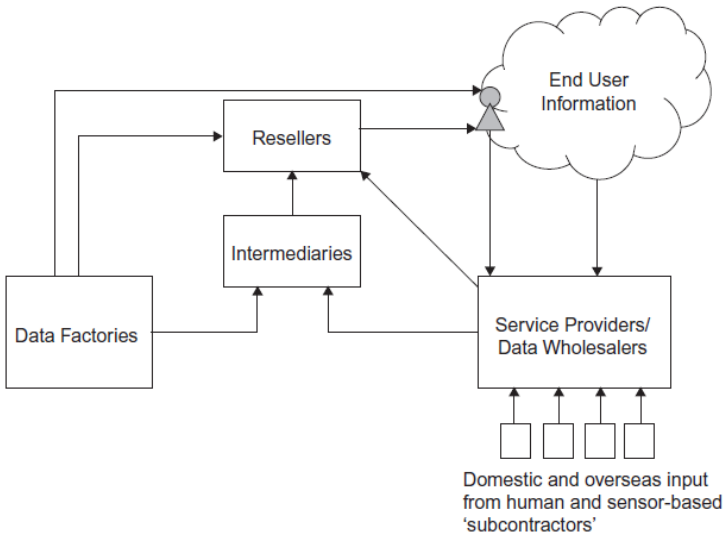


Fig.2.10: Information Driven Global Value Chain

2.9.3. Production flow in I-GVC

Domestic/overseas inputs (sensors + humans) → Data Factories/Wholesalers → Intermediaries (privacy, specialization) → Resellers (analytics + packaging) → Information Products for firms/end-users.

2.10. Global Information Monopolies

2.10.1. What are they?

- Firms that dominate the collection, processing, and monetization of information within the I-GVC, often by controlling platforms, data access, and network effects.
- They capture market power via intangible assets (data, algorithms, platforms) and centralization of network externalities.

2.10.2. How monopolistic power arises in I-GVC

- Data network effects: more users → more data → better services → more users (self-reinforcing).
- Platform control: owners of major platforms (search, social, ad-tech, cloud) set terms for data exchange and intermediaries.
- High switching costs: ecosystems lock in developers, advertisers, and enterprises.

2.10.3. Implications

- Market concentration in information products (ads, insights, targeting).
- Privacy and governance challenges (consent, anonymization, regional laws).
- Potential for "information marketplaces" to be skewed toward a few dominant orchestrators.

2.11. IoT Architecture components

IOT Architectures are having following components:

1. Sensors/Actuators: These are the devices that are able to emit, accept and process data over the network. These sensors or actuators may be connected either through wired or wireless. This contains GPS, Electrochemical, Gyroscope, RFID, etc. Most of the sensors need

connectivity through sensors gateways. The connection of sensors or actuators can be through a Local Area Network (LAN) or Personal Area Network.

2. Gateways and Data Acquisition: As the large numbers of data are produced by these sensors and actuators need the high-speed Gateways and Networks to transfer the data. This network can be of type Local Area Network (LAN such as WiFi, Ethernet, etc.), Wide Area Network (WAN such as GSM, 5G, etc.).
3. Edge IT: Edge in the IoT Architecture is the hardware and software gateways that analyze and pre-process the data before transferring it to the cloud. If the data read from the sensors and gateways are not changed from its previous reading value then it does not transfer over the cloud, this saves the data used.
4. Data center/ Cloud: The Data Center or Cloud comes under the Management Services which process the information through analytics, management of device and security controls. Beside this security controls and device management the cloud transfers the data to the end user's application such as Retail, Healthcare, Emergency, Environment, and Energy, etc.

2.12. Comparing IoT Architectures

- Architectures is supporting data, process, and the functions that endpoint devices perform.
- Two of the best-known architectures
    - **oneM2M**
    - **IoT World Forum (IoTWF)**

2.12.1. The oneM2M IoT Standardized Architecture

European Telecommunications Standards Institute (ETSI) created the M2M Technical Committee in 2008. The goal of this committee is to create a common architecture that would help accelerate the adoption of M2M applications and devices.

- One of the greatest challenges in designing an IoT architecture is dealing with the heterogeneity of devices, software, and access methods.

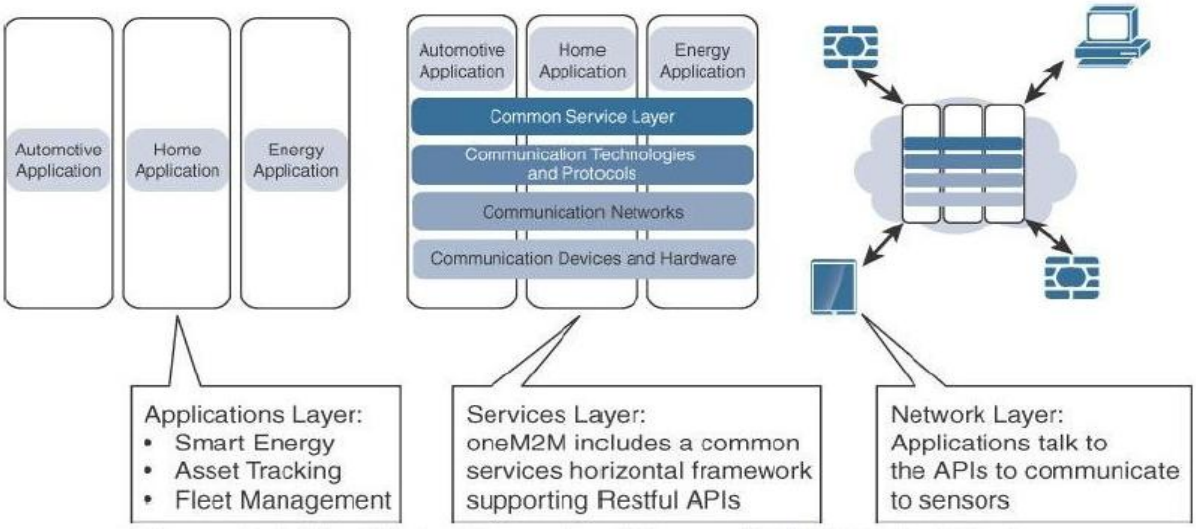


Fig.2.11: One M2M IoT Architecture

The oneM2M architecture divides IoT functions into three major domains:

1. Application layer

   The oneM2M architecture gives major attention to connectivity between devices and their applications. This domain includes the application-layer protocols and attempts to standardize API with definitions for interaction business intelligence (BI) systems.

2. Services layer

   This layer is shown as a horizontal framework across the vertical industry applications. At this layer, horizontal modules include the physical network that the IoT applications run on, the underlying management protocols, and the hardware.

3. Network layer

   This is the communication domain for the IoT devices and endpoints.

   It includes the devices themselves and the communications network that links them. The communications infrastructure includes wireless mesh technologies, such as IEEE 802.15.4, and wireless point-to-multipoint systems, such as IEEE 801.11. Also included are wired device connections, such as IEEE 1901 power line communications.

2.12.2. The IoT World Forum (IoTWF) Standardized Architecture

In 2014 the IoTWF architectural committee (led by Cisco, IBM, Rockwell Automation, and others) published a seven-layer IoT architectural reference model. This architecture is to visualizing IoT from a technical perspective. Each of the seven layers is broken down into specific functions, and security encompasses the entire model.

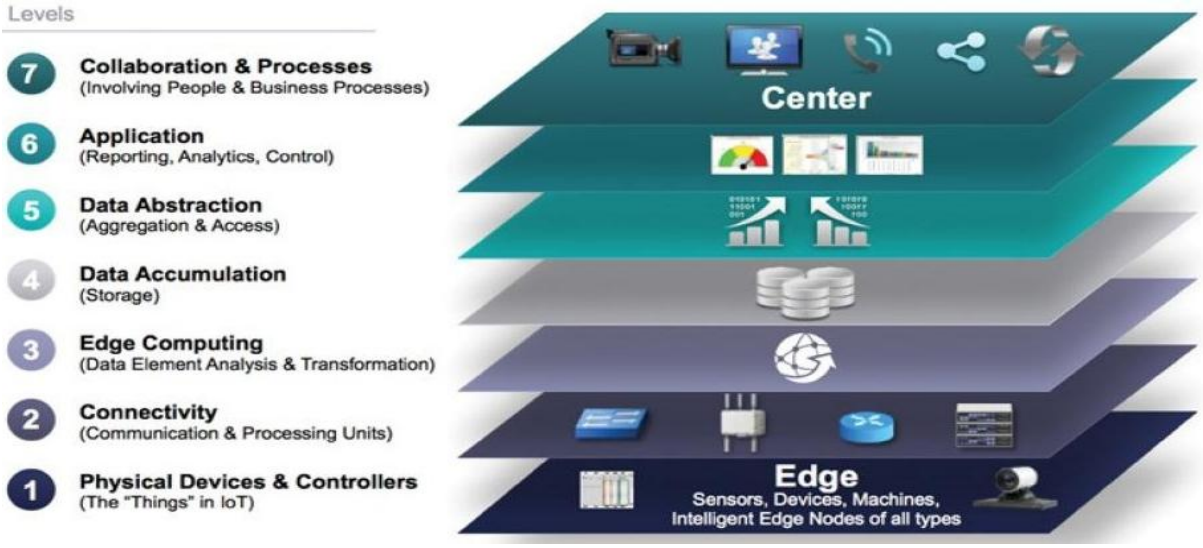


Fig.2.12: IoTWF Architecture

2.13. A simplified IoT Architecture

- This framework is presented as two parallel stacks:
  - i. The Core IoT Functional Stack
  - ii. The IoT Data Management and Compute Stack

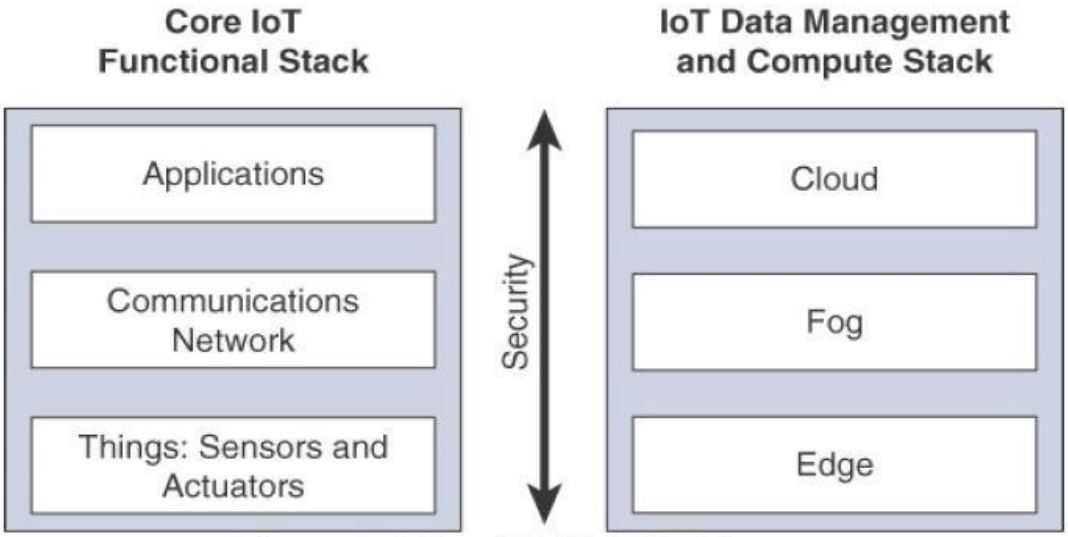


Fig.2.13: Simplified IoT Architecture

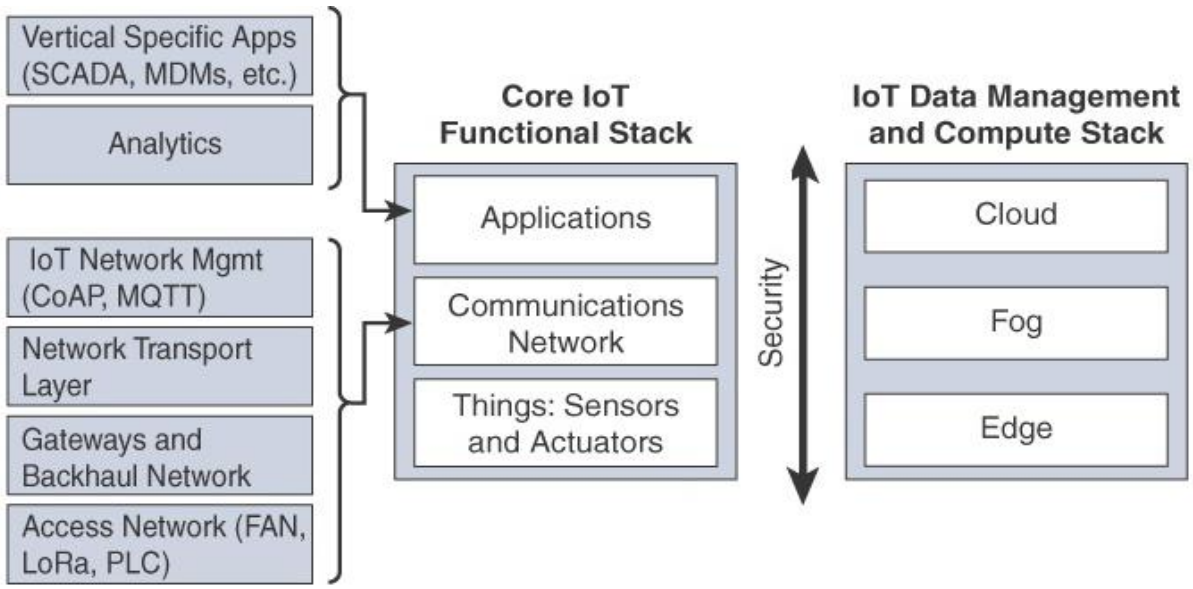


Fig.2.14: Expanded View of Simplified IoT Architecture

2.13.1. Core IoT Functional Stack

Core IoT Functional stack has 3 layers:

1. Things layer
2. Communications network layer
    - Access network sublayer
    - Gateways and backhaul network sublayer
    - Network transport sublayer
    - IoT network management sublayer
3. Application and analytics layer

Layer1 Things: Sensors and Actuators

Most IoT networks start from the object, or "thing," that needs to be connected.

These objects are "smart" because they use contextual information and configured goals to perform actions.

A "thing" interacts with an external system to report data, exchange it with other objects, or interact with a management platform.

There are six ways to classify a smart object, they are:

1. Power- Battery-powered or line-powered
2. Mobility- Mobile or static placement
3. Frequency- Low or high reporting frequency

4. Data Richness- Simple or rich data per report
5. Report Range- Distance to the gateway
6. Density- Objects per cell, per gateway

Layer 2: Communications Network Layer

When smart objects aren't self-contained, they must communicate with an external system — often over wireless. This layer has four sublayers

i. Access Network Sublayer- How the thing reaches its gateway
   a. Range Categories: Range between the smart object and the information collector drives the choice of access technology.
      PAN, HAN, NAN, FAN, LAN..etc
   b. Communication Topologies: point to point, star, mesh…

ii. Gateways & Backhaul Sublayer- Bridging to the central station
   Data collected from a smart object often needs to be forwarded to a central station in a different location. The gateway handles this inter-medium communication — moving data from the access technology onto the backhaul.
   The smart objects may be static or mobile within a limited area; the gateway itself is usually static.
   Example: Ethernet Backhaul, WiMAX (802.16)

iii. Network Transport Sublayer- TCP / UDP — reliability vs. speed
   Responsible for end-to-end data transfer — reliability, flow control, and error handling.

iv. IoT Network Management Sublayer- Lifecycle, security, optimization
   Oversees and controls the network of connected devices so they operate smoothly and securely.
   a. Device Lifecycle- Provisioning, configuration, monitoring, firmware updates, decommissioning
   b. Security & Access- Authentication, authorization, key management, anomaly detection
   c. Network Optimization- Quality of Service, load balancing, fault tolerance & recovery

   Example: CoAP, MQTT…

Layer 3: Applications and Analytics Layer

The topmost layer — where collected data is processed, analyzed, and turned into business or user value.

i. Applications:
   a. User Interface — dashboards, mobile apps, portals
   b. Business Logic — the rules that drive the system
   c. Device Control — commands sent to lower layers

ii. Analytics
   a. Descriptive — what has happened
   b. Predictive — what is likely to happen next
   c. Prescriptive — what action to take

2.13.2. IoT Data Management and Compute Stack

- It is responsible for handling, processing, and storing the immense volume of data generated by connected devices.
- It's the engine that transforms raw data into a structured format, ready for analysis and application.

i. IoT Data Lifecycle:

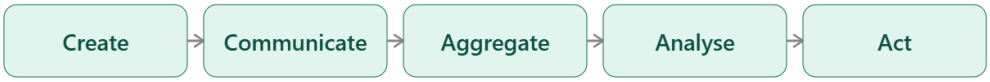


Fig.2.15: IoT data Life Cycle

Phase	What Happens	Example
Create	Sensors collect raw data	Temperature = 25°C
Communicate	Data transmitted via network	MQTT/HTTP to gateway
Aggregate	Filter, compress, summarize	Avg temp every 5 min
Analyse	Extract patterns/insights	Detect anomalies
Act	Take action based on analysis	Turn on AC if temp > 30°C

ii. Compute stack:
   - Edge Computing: Processing on/near the device (ultra-low latency)
   - Fog Computing: Intermediate layer at gateways (reduces cloud traffic)
   - Cloud Computing: Centralized processing in data centers (high compute power)

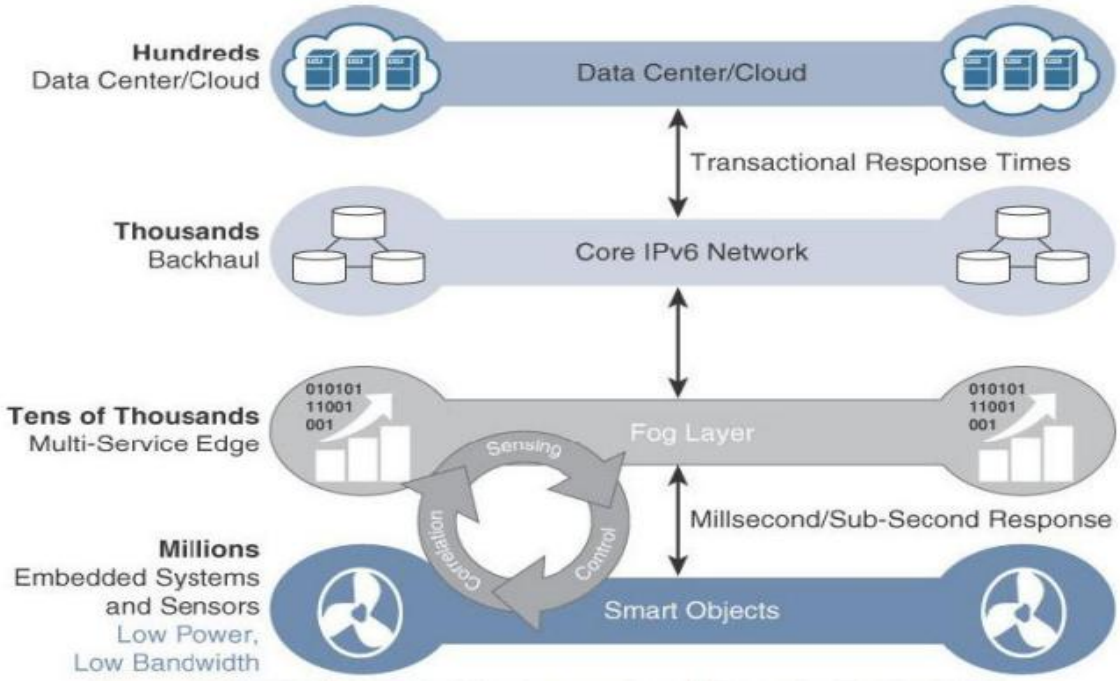


Fig.2.16: IoT Data Management and Compute stack with Fog computing

Comparison of Edge, Fog, Cloud:

Feature	Edge	Fog	Cloud
Location	On device	Gateway	Data center
Latency	< 50 ms	50–500 ms	> 5000 ms
Power	Very low	Moderate	Very high
Use Case	Emergency control	Local analytics	ML training

Data Management Challenges

- Volume: Billions of devices → massive data
- Velocity: High-speed streaming data
- Variety: Different formats (JSON, XML, binary)
- Veracity: Sensor noise, missing values
- Storage: Need time-series databases + indexing
- Querying: Real-time + historical queries
- Integration: Multiple protocols (MQTT, CoAP, HTTP)